

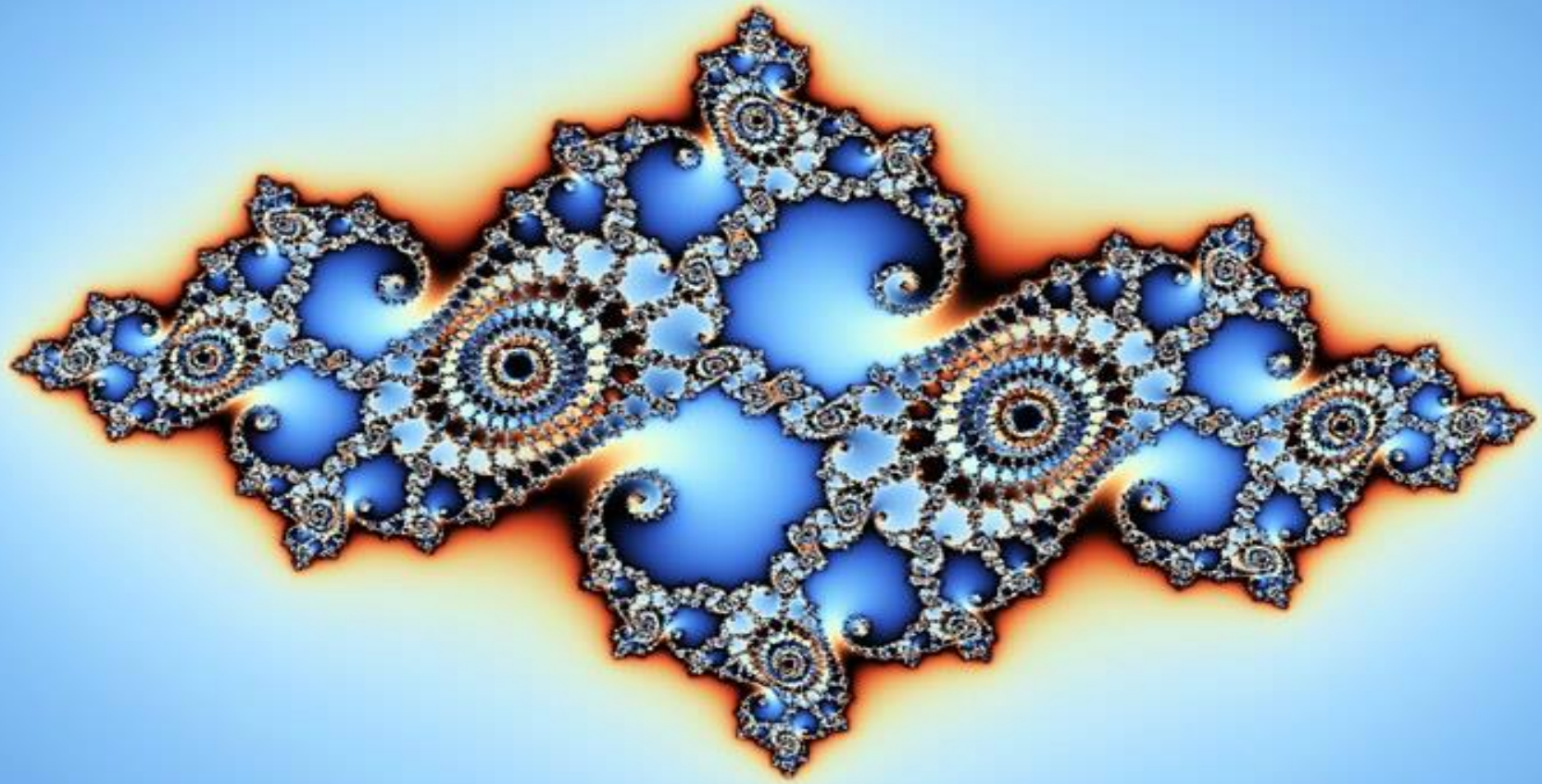
OOP2 KU

Fractals, GLSL, LWJGL, GLFW and Java GPU Rendering

Sebastian David Albu, WS 2024

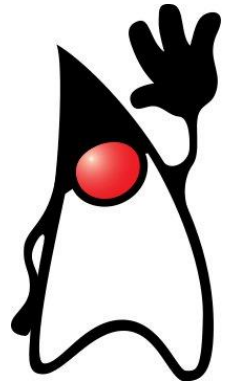
Based on slides from WS 2021 and WS 2022

Institute of Computer Graphics and Knowledge Visualization (CGV)

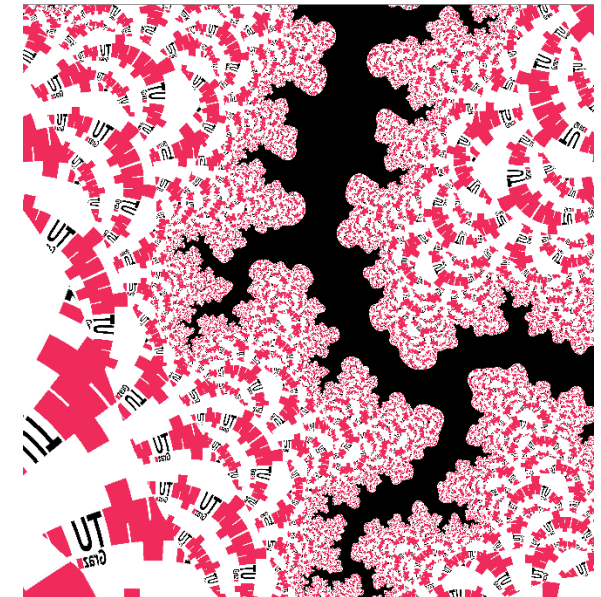
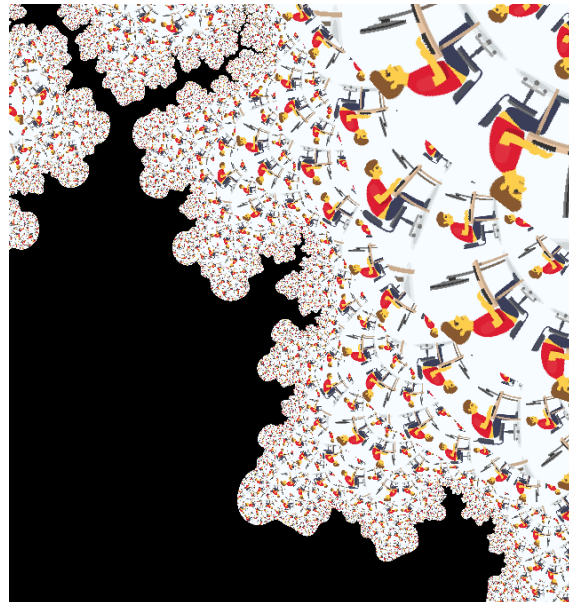


Introduction and Math Recap

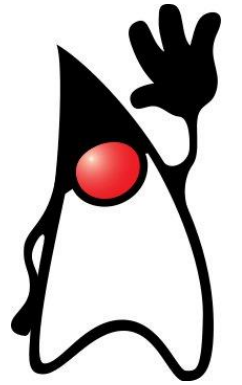
Up Front...



- Fractal sets will not be the main focus of exam interviews
- We will treat them very informally in this presentation
- They will just be a means to create interesting pictures procedurally
 - *Spoiler:*
very little code needed!
 - (Entire reference solution for bonus assignment has < 300 LOC) !!

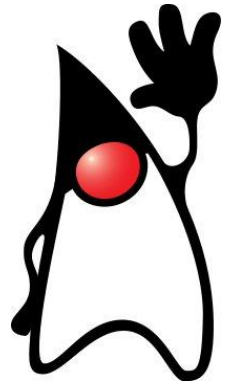


Up Front...

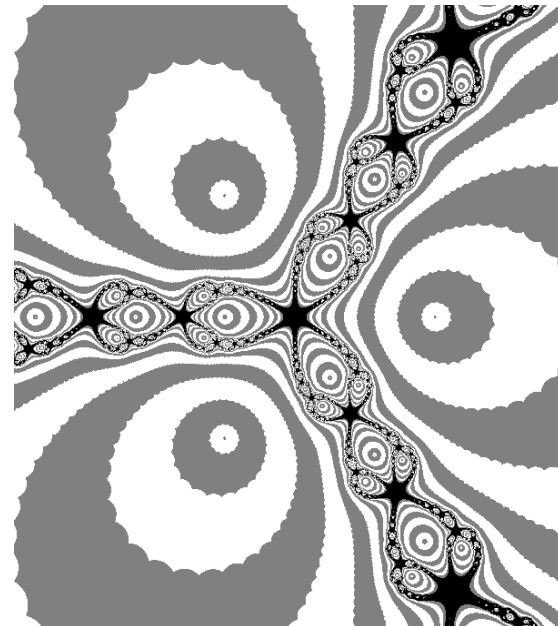
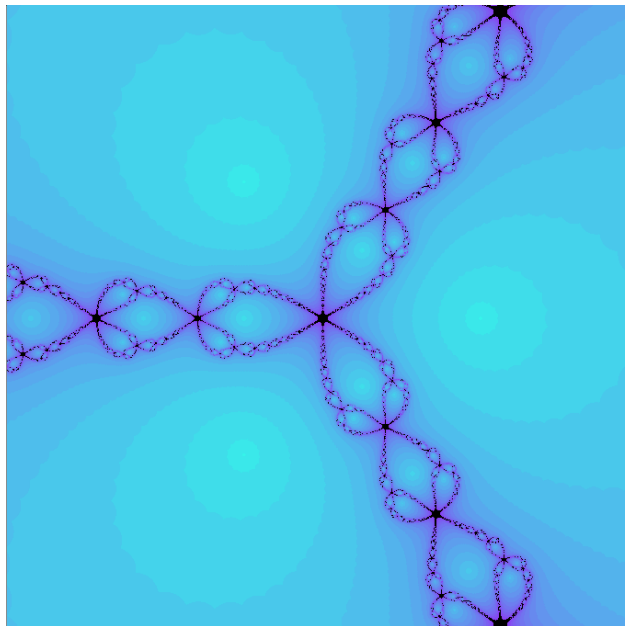
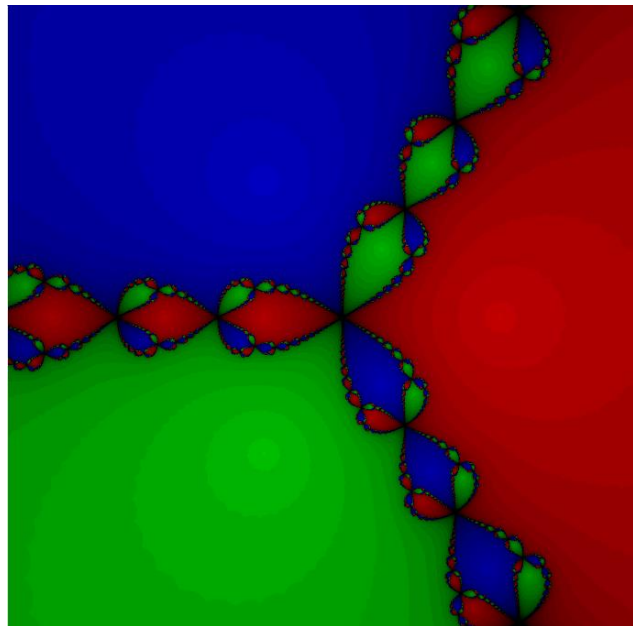


- The bonus assignment has a very length README
- BUT – everything is explained step by step
- **The entire reference has < 300 LOC**
 - **You only need to implement ~100 LOC**
- Every step is documented and every formula is given
- Just read a bit into GLSL for the shaders
 - GLSL: 2 LOC for Vertex Shader, 20 LOC for Fragment Shader
 - (Rest should all be explained in detail – Java Syntax)
- GLFW functions are all linked in the README
- The README should be more than sufficient – Give it a try ☺

Fractal Sets – Newton Raphson Fractal



- Named after Sir Isaac Newton and Joseph Raphson
- Great introduction to the world of fractals (see Link below)



Sir Isaac Newton



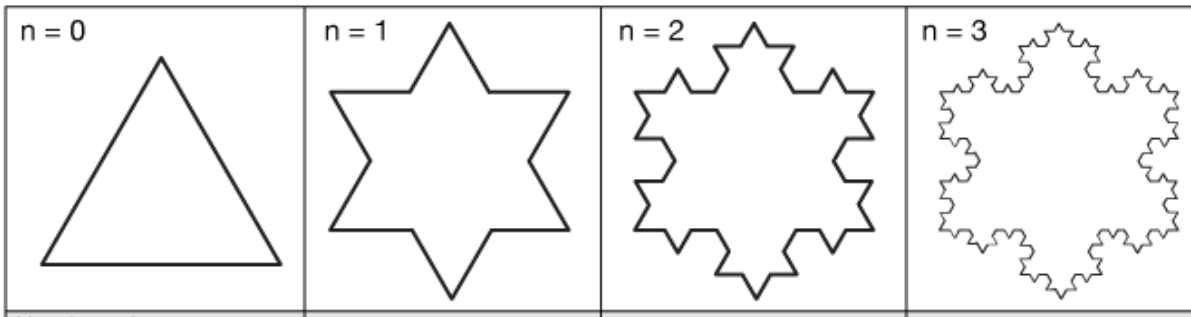
This Photo by Unknown Author is licensed under [CC BY-NC-ND](#)

Recommended:

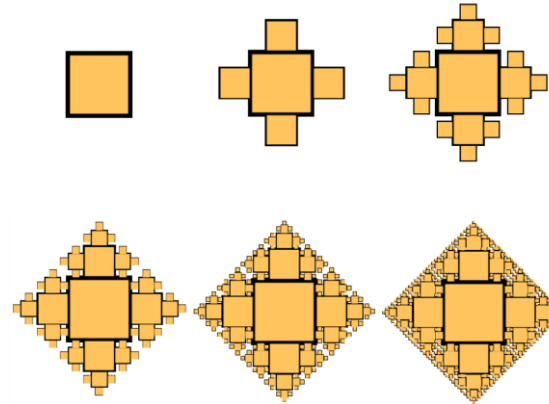
“Newton’s method, and the fractal it creates that Newton knew nothing about” by 3Blue1Brown

<https://www.youtube.com/watch?v=-RdOwhmqP5s>

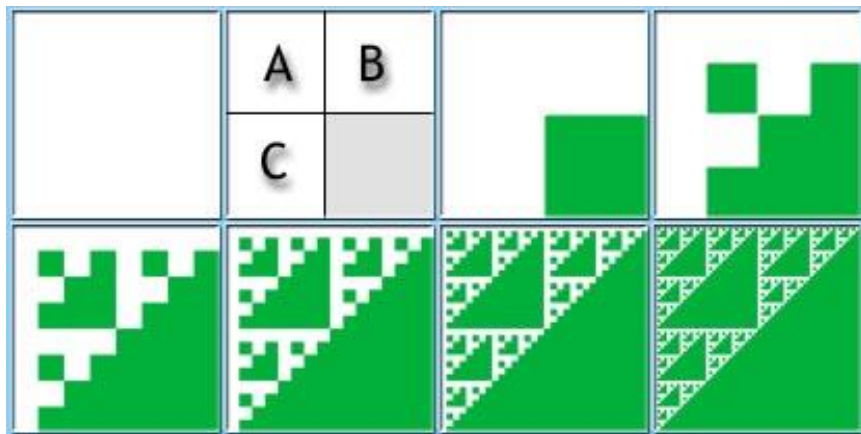
Fractal Curves & Iterated Function Systems



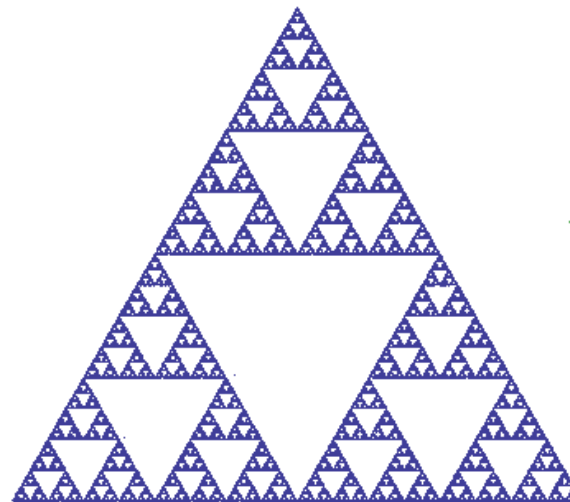
[This Photo](#) by Unknown Author is licensed under [CC BY-SA-NC](#)



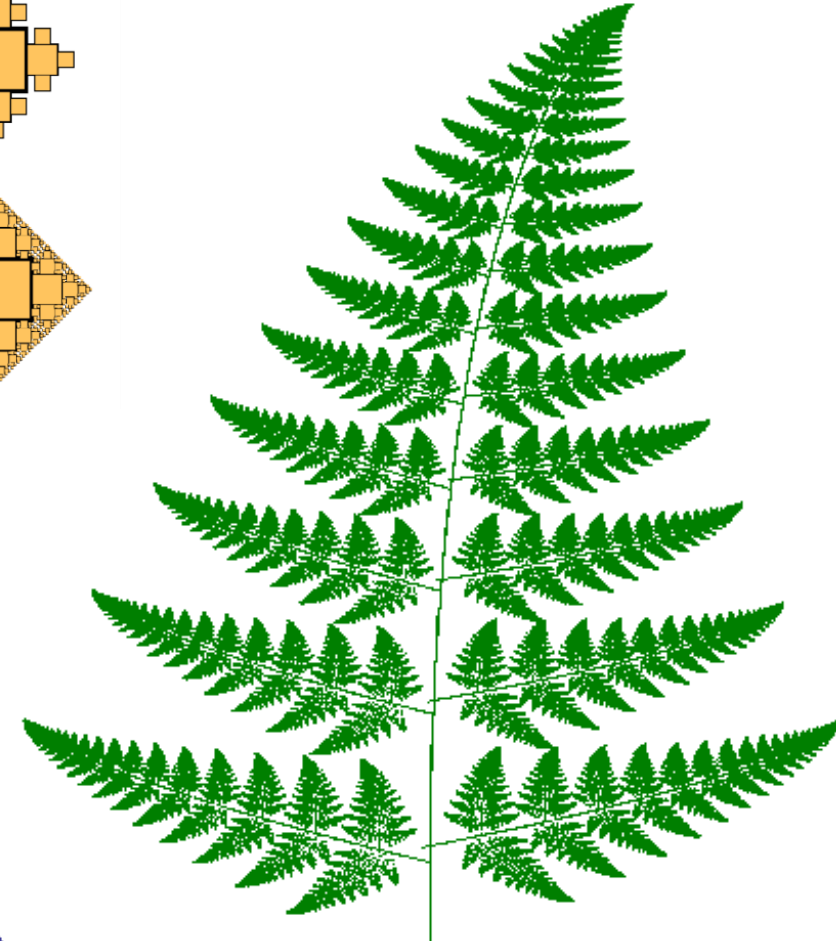
[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)



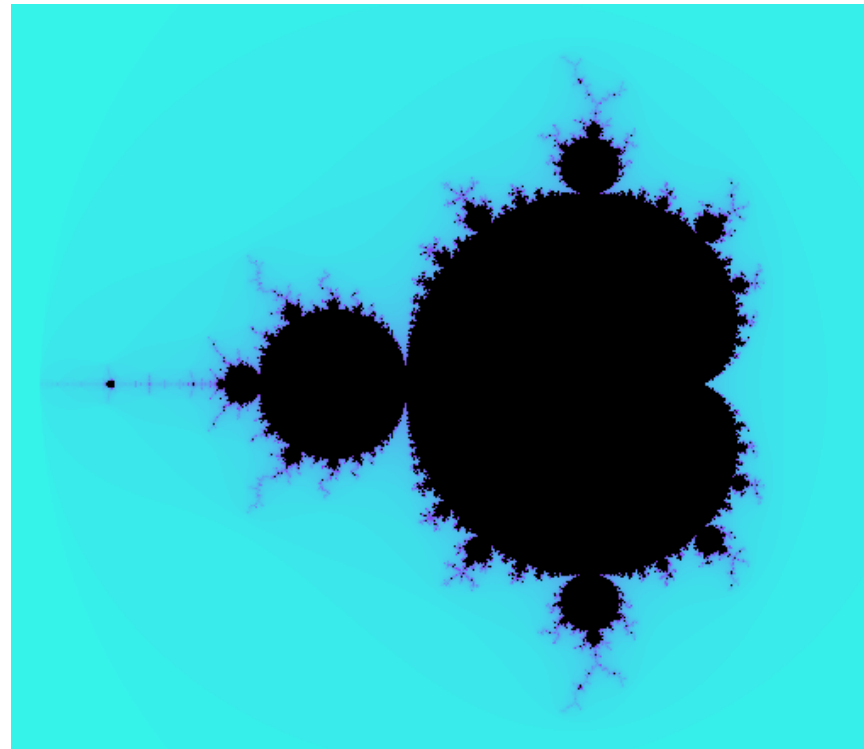
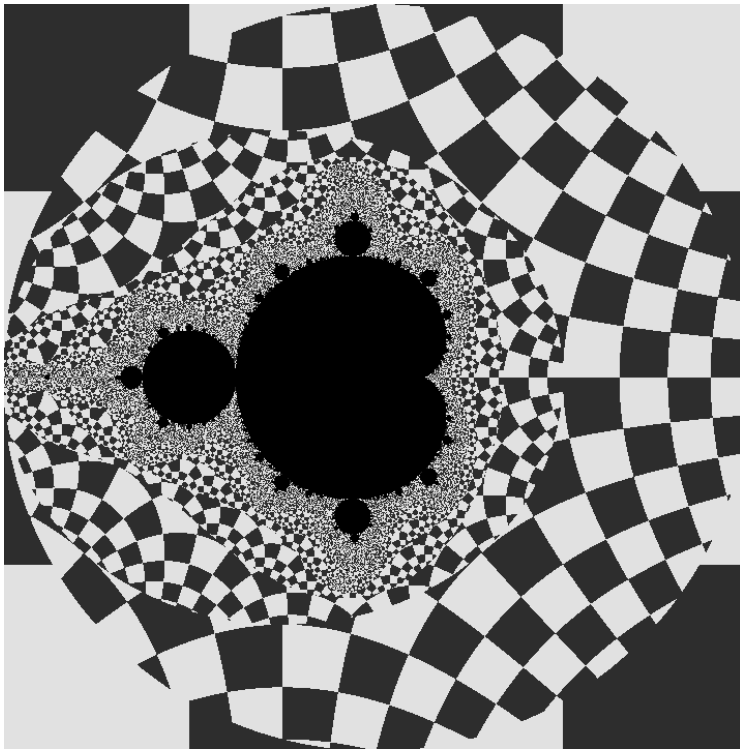
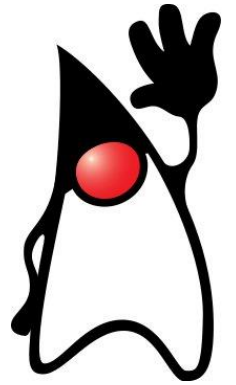
[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

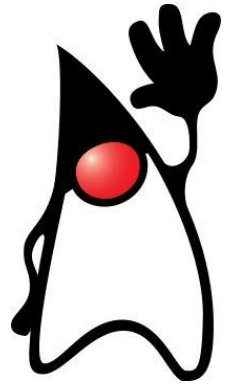
Fractal Sets – Mandelbrot Set

- Named after Benoit Mandelbrot

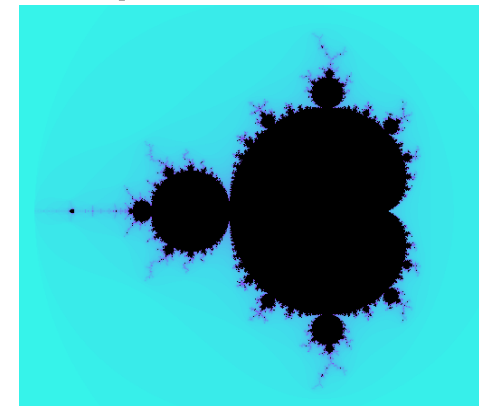


Benoit Mandelbrot
(1924-2010)

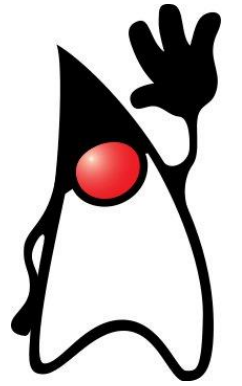
Fractals used in the KU



- We will only calculate and display the Mandelbrot set
 - We will slightly modify the equations to keep things interesting
 - Details are given in assignment descriptions in GitLab repos
 - Slides only show the “base” fractal sets explicitly
- The fractal set is defined on the *complex plane*
 - Plane that contains all complex numbers
 - This will be defined in the vertex shader (**complexPlane.vert**)

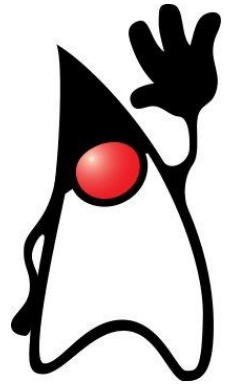


Complex Numbers - Refresher



- Typically written as: $z = a + b * i$
- Consist of *real part* a and *imaginary part* b
 - Both parts are "standard" real numbers for example, $z = 69 + 420 * i$
- i is the imaginary unit, and $i^2 = -1$
 - (this means that we can calculate the square root of negative numbers)

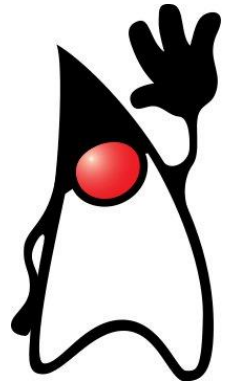
Complex Numbers - Refresher



- Let z_1 and z_2 be two complex numbers
 - $z_1 = a_1 + b_1i$
 - $z_2 = a_2 + b_2i$
- Adding complex numbers: add real parts and imaginary parts
 - $z_1 + z_2 = (a_1 + a_2) + (b_1 + b_2)i$
- Multiplying complex numbers
 - $z_1 * z_2 = (a_1a_2 - b_1b_2) + (a_1b_2 + a_2b_1)i$

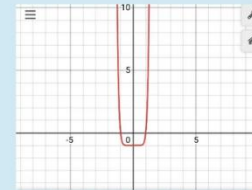
This may work easier as well, right?

Complex Numbers - Refresher



STOP USING COMPLEX NUMBERS

- Yes we get it, $i^2 = -1$. And?????
- "Trust me guys this order 10 polynomial has 10 roots" Like look at the graph is your brain malfunctioning???
- These are the ways people represent complex numbers:

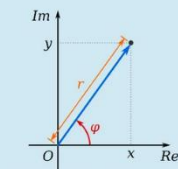


DOES THIS LOOK LIKE 10 ROOTS TO YOU?

$$i \equiv \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \quad |z| (\cos \theta + i \sin \theta)$$

A MATRIX???

YEAH THIS MAKES A LOT OF SENSE



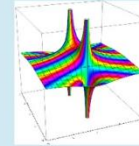
USING A GRAPH NOW???

$$z = re^{i\theta}$$

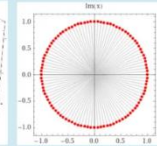
BLATANTLY STEALING
EULER'S FORMULA

$$a + ib$$

Ok this one actually
makes sense



HUH????



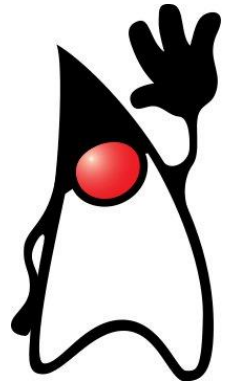
?????????

"Hi can I have $3i$ apples please"

"I am  years old"

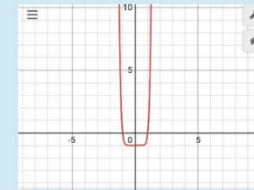
THEY HAVE PLAYED US FOR
ABSOLUTE FOOLS

Complex Numbers - Refresher



STOP USING COMPLEX NUMBERS

- Yes we get it, $i^2 = -1$. And?????
- "Trust me guys this order 10 polynomial has 10 roots" Like look at the graph is your brain malfunctioning???
- These are the ways people represent complex numbers:

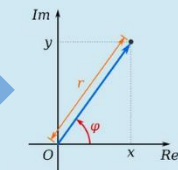


DOES THIS LOOK LIKE 10 ROOTS TO YOU?

$$i \equiv \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \quad |z| (\cos \theta + i \sin \theta)$$

A MATRIX???

YEAH THIS MAKES A LOT OF SENSE



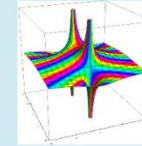
USING A GRAPH NOW???

$$z = re^{i\theta}$$

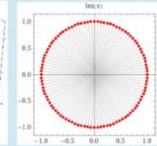
BLATANTLY STEALING EULER'S FORMULA

$$a + ib$$

Ok this one actually makes sense



HUH????



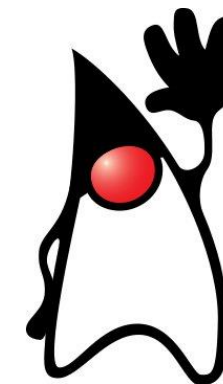
?????????

"Hi can I have $3i$ apples please"

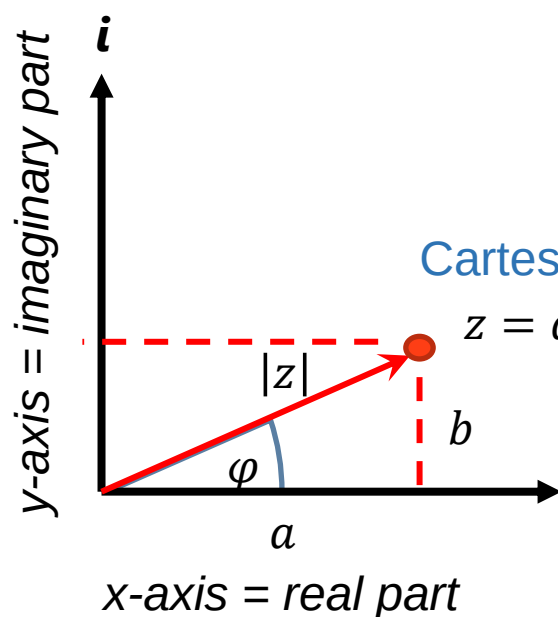
"I am  years old"

THEY HAVE PLAYED US FOR ABSOLUTE FOOLS

Cartesian Form Polar Form



- We can think of a complex number as a 2D coordinate on the Cartesian complex plane



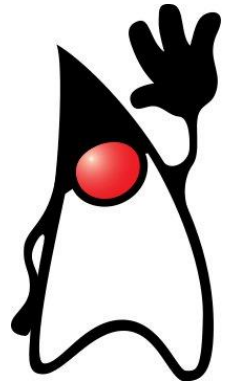
Polar Form: $|z| = \sqrt{a^2 + b^2}$

$$\varphi = \begin{cases} \tan^{-1}\left(\frac{b}{a}\right) \dots \text{if } a \geq 0 \\ \tan^{-1}\left(\frac{b}{a}\right) + \pi \dots \text{if } a < 0 \end{cases}$$

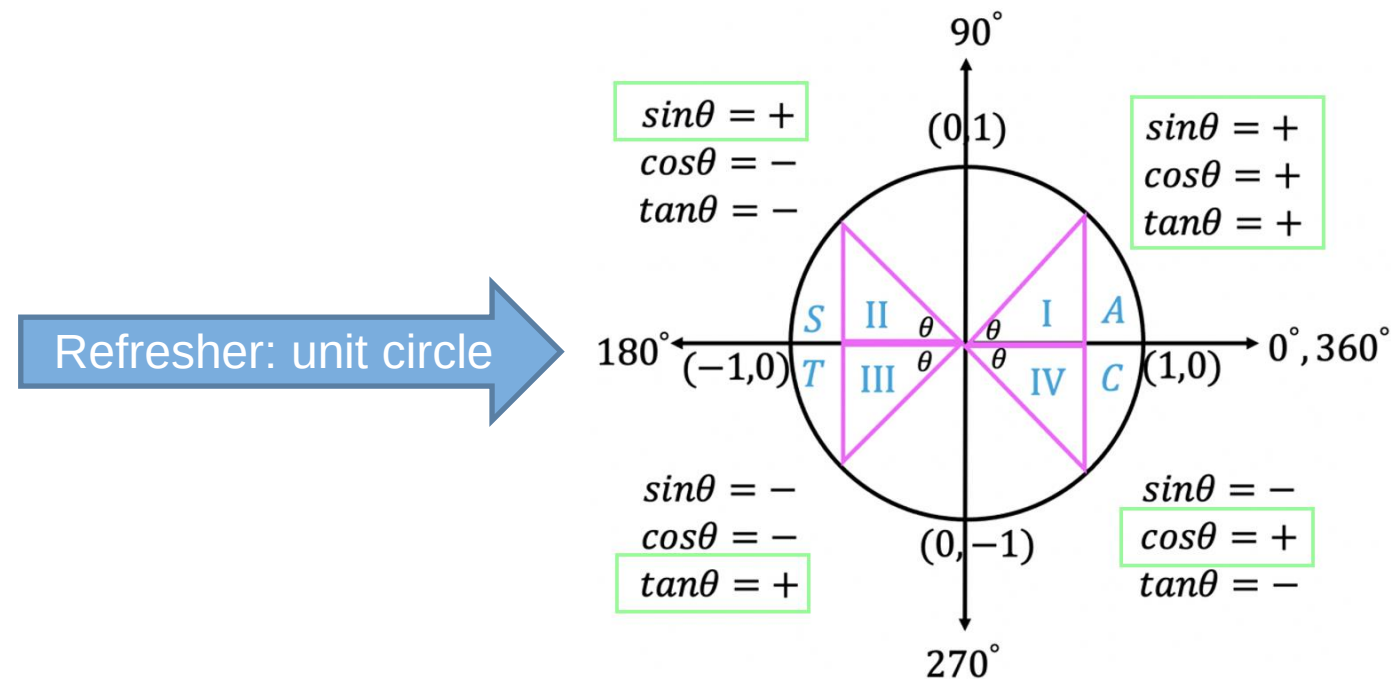
Alternative ways
to calculate
(sign checks left as an
exercise)

$$\begin{cases} \cos(\varphi) = \frac{a}{|z|} \Leftrightarrow \varphi = \cos^{-1}\left(\frac{a}{|z|}\right) \\ \sin(\varphi) = \frac{b}{|z|} \Leftrightarrow \varphi = \sin^{-1}\left(\frac{b}{|z|}\right) \end{cases}$$

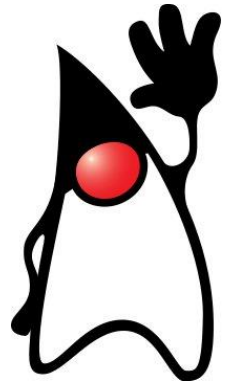
Polar Form – Pitfalls



- Pay attention to the quadrants the complex number lies in
 - Sanity check calculated polar-form angle
 - Know your units (DEG vs. RAD)!

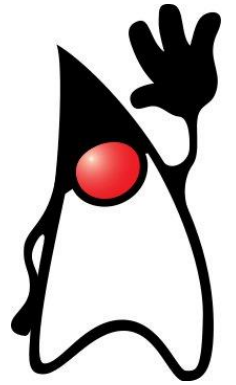


Polar Form vs. Cartesian Form

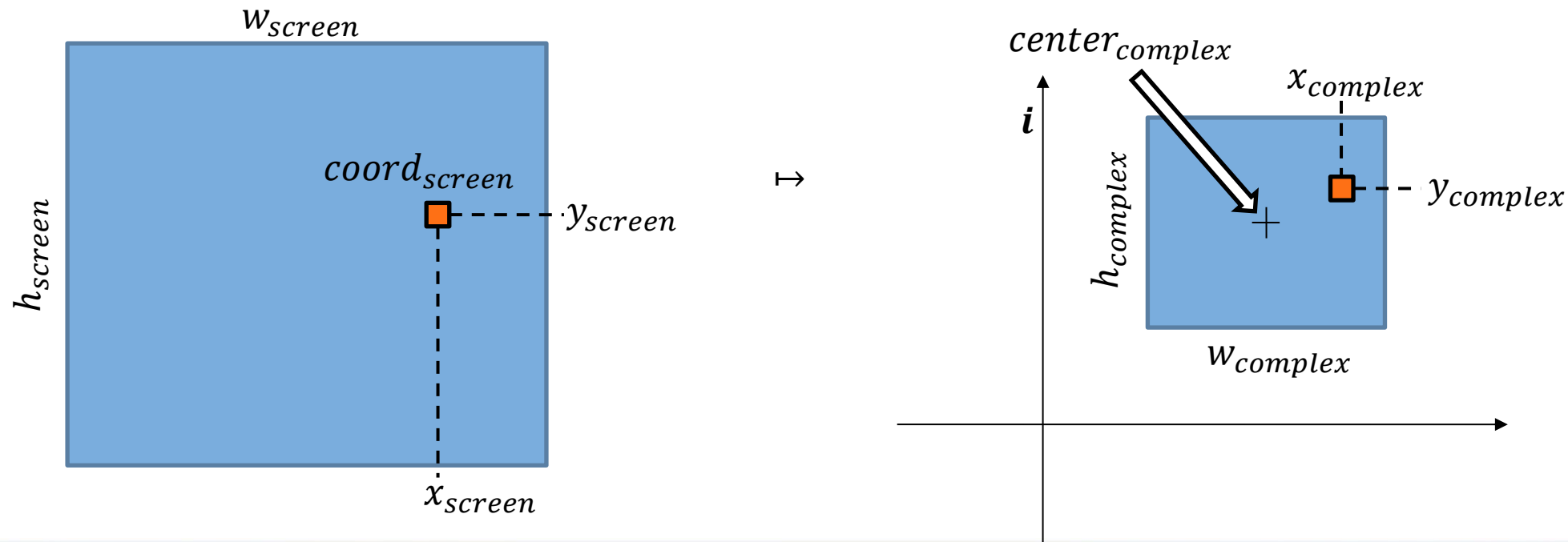


- Note that **both** cartesian form and polar form **are 2D coordinates**
- Cartesian form:
 - **2 real numbers** that denote distances along the x-axis and the y-axis
- Polar form:
 - **2 real numbers** that denote the magnitude (= Length) and the angle of the resulting point on the complex plane
 - Magnitude = distance to origin
 - Angle = measured from positive x-axis in counter-clockwise direction just as in unit-circle

Mapping Pixel Coords to Complex Numbers



- Map the pixel to a rectangular area of size on the complex plane centered about the complex number, where the relative position of the pixel on the screen is preserved within the rectangular area on the complex plane



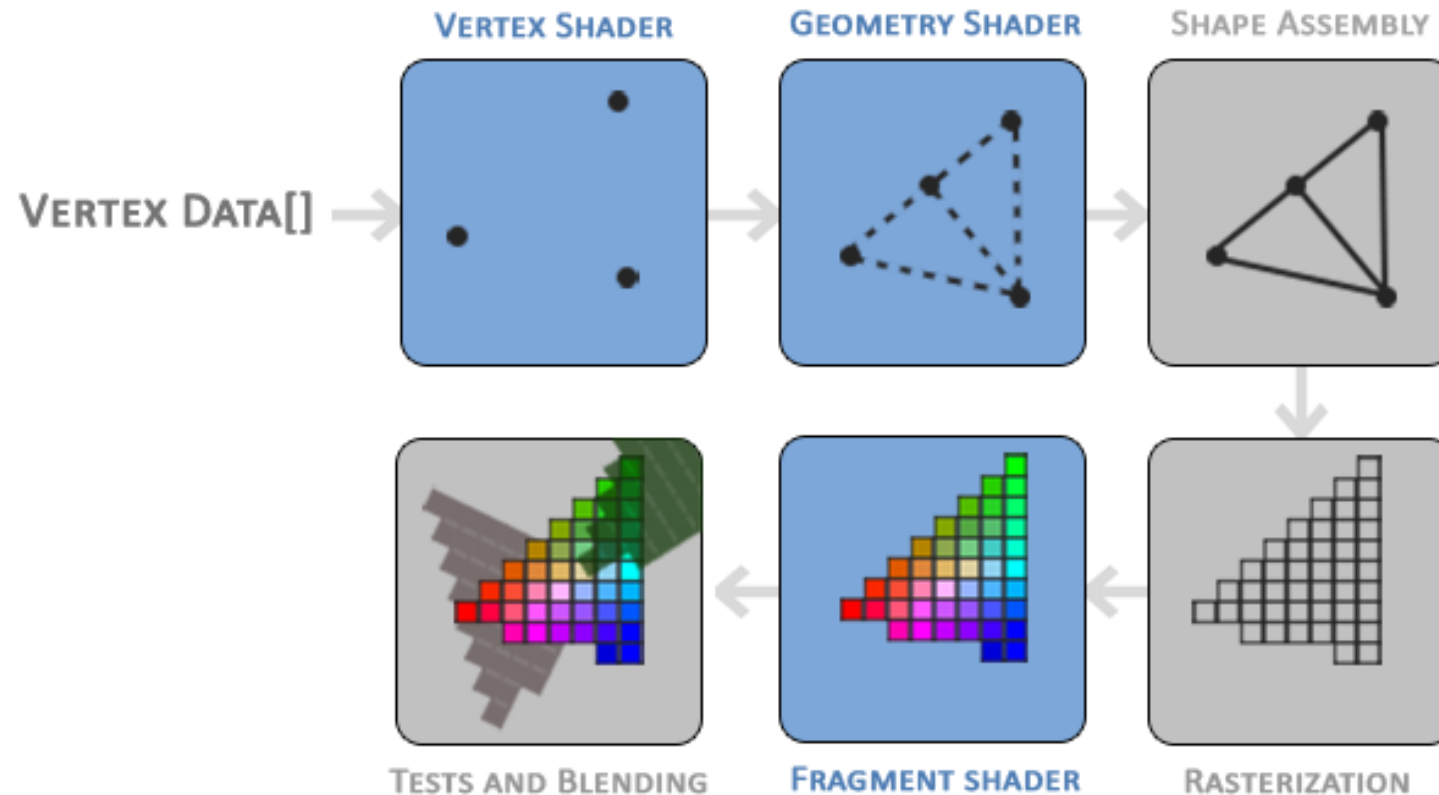
Introduction to Shaders

Understanding Shaders



- Vertex Shader:
 - Transforms vertex data (like positions) from object space to screen space.
- Fragment Shader:
 - Determines pixel color by applying shading, texturing, and other effects.
- Use of GLSL (OpenGL Shading Language) to write shader code.

Understanding Shaders



Understanding Shaders (GLSL)



- GLSL is a C-like language with specialized types for graphics programming.
- **vec4**: A 4-component vector (e.g., RGBA colors, 3D points with a w-component for perspective).
 - Example: `vec4(1.0, 0.5, 0.0, 1.0)` could represent an orange color with full opacity.
 - Used for position, color, and more.
- **vec3**, **vec2**: 3- and 2-component vectors (e.g., for 3D/2D coordinates or texture data).
 - Example: `vec3(1.0, 0.0, 0.0)` represents the red color. (RGB) → only 1 set for RED
- **mat4**: A 4x4 matrix (commonly for transformations like scaling, rotating, and projecting vertices).
 - Example: `uniform mat4 projection;` stores a projection transformation.
- Precision Specifiers:
 - GLSL supports **float**, **int**, and **bool**, just like C.
 - float values often represent coordinates, colors, and other continuous values.

Understanding Shaders (GLSL)



- How shaders share data:
 - **uniform:**
 - Read-only data shared across all vertices/fragments.
 - Example: uniform float power; stays constant during rendering.
 - Used for scene-wide parameters (e.g., transformation matrices, lighting settings).
 - **attribute** (in vertex shaders):
 - Per-vertex input data (e.g., position or texture coordinates).
 - Example: attribute vec2 position; for 2D vertex positions.
 - **varying:**
 - Passes interpolated data from vertex shader to fragment shader.
 - Example: varying vec2 c; provides coordinates from the vertex to the fragment shader.
 - Useful for smooth gradients, texture mapping, etc..

Vertex Shader of the Complex Plane



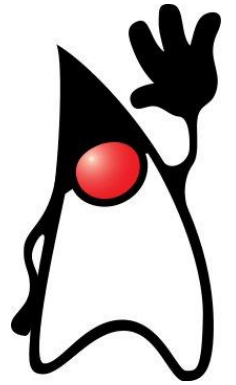
- Calculation in the **complexPlane.vert** file
 - Using GLSL syntax for both vertex and fragment shaders
 - This vertex shader is used to map the vertices of the complex plane to screen space.
 - It also passes the position of each vertex as a varying variable to the fragment shader.
- **Key Variables:**
 - **attribute vec2 position:** Represents the 2D position of each vertex in the complex plane.
 - **uniform mat4 projection:** A 4x4 matrix used to transform vertex coordinates from the complex plane to the screen space.
 - **varying vec2 c:** Passes the position of each vertex to the fragment shader for further processing.
 - **gl_Position:** A built-in vec4 in the vertex shader specifies a vertex's final position in clip space.

Transformation and Output in Vertex Shader

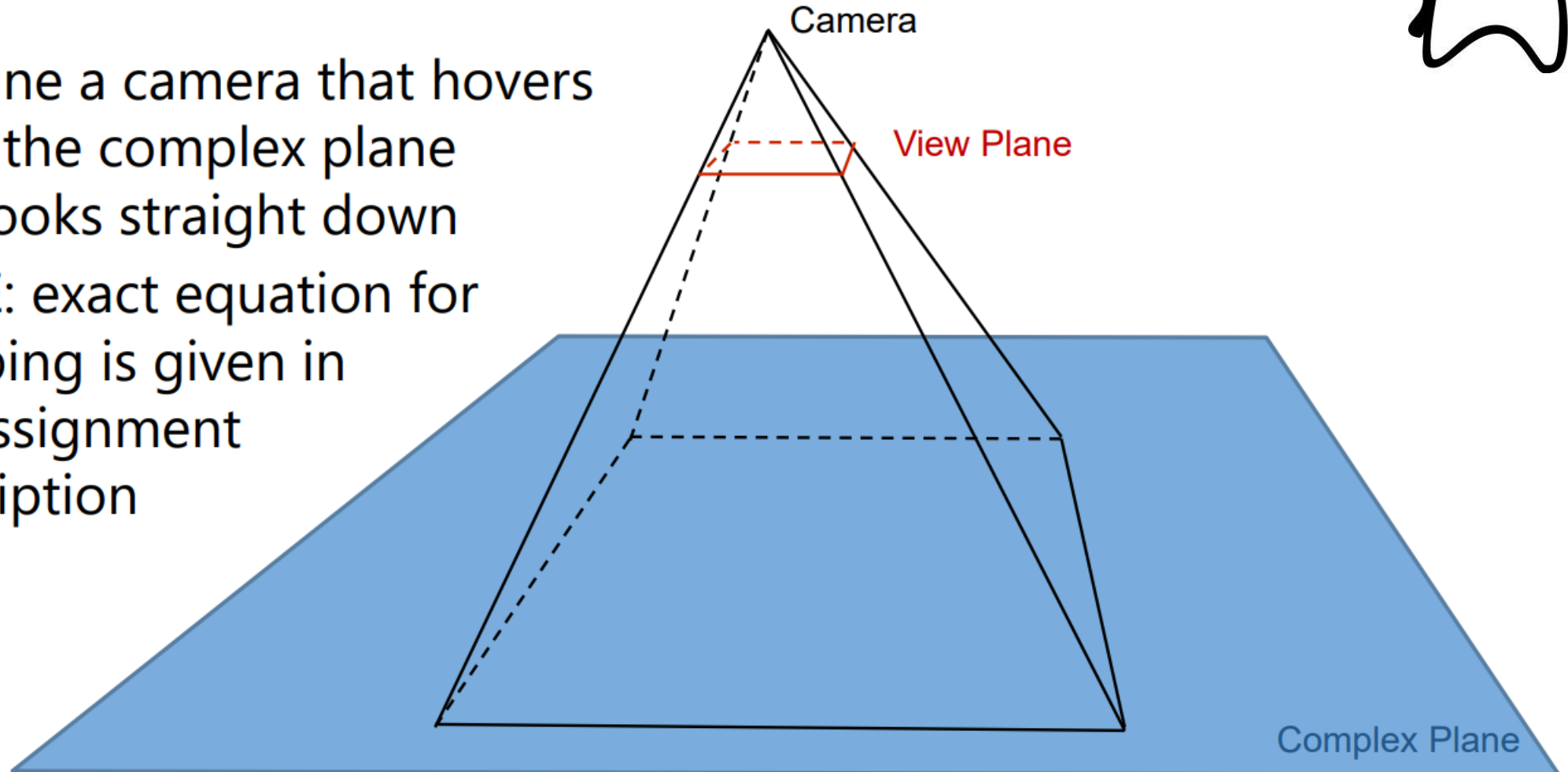


- Calculation in the **complexPlane.vert** file
- Transformation:
 - Converts the 2D position into a 4D vector with a z value of 0 and a w (homogeneous coordinate) of 1.
 - Multiplies this vector by the projection matrix to transform it into screen coordinates.
- Output (Passing to Fragment Shader)
 - Directly assigns the vertex position to c , which will be used in the fragment shader to determine the color of the corresponding pixel.

Mapping Pixel Coords to Complex Numbers



- Imagine a camera that hovers over the complex plane and looks straight down
- NOTE: exact equation for mapping is given in the assignment description



Calculating the Mandelbrot Set



- Fragment Shader
 - This fragment shader is used in the rendering pipeline to calculate and visualize fractals, such as the Mandelbrot set.
 - This specific shader uses complex number arithmetic and iterative methods to determine the color of each pixel based on its position in the complex plane.
 - → We first need the vertex shader for the complex plane

Calculating the Mandelbrot Set



- Fragment Shader – Key Variables of the **.frag** file
- Note that we use GLSL syntax
- Uniforms:
 - **power**: Controls the exponentiation in the complex number calculations.
 - **maxIterations**: Determines the maximum number of iterations before a point is considered to belong to the Mandelbrot set.
- Varying:
 - **c**: The complex coordinate corresponding to the current pixel being processed.

gl_FragColor: A built-in vec4 in the fragment shader that defines the output color of a pixel on the screen.

Calculating the Mandelbrot Set



- Complex Number Exponentiation
 - Note that we use GLSL syntax.
- **vec2 complexPow(in vec2 z, in float k)**
 - This function computes z^k , where z is a complex number represented by a vec2.
 - Calculate the argument (angle) of the complex number using atan(...) in GLSL.
 - Calculate the magnitude (length) of the complex number. (GLSL: length(z))
 - Raise the magnitude to the power of k.
 - Multiply the argument by k.
 - Return the new complex number in polar coordinates, converting it back to cartesian form with cos and sin. → **mag * vec2(cos(angle), sin(angle))**

Calculating the Mandelbrot Set



- Mandelbrot Iteration
 - Note that we use GLSL syntax.
- Can be implemented directly in **void main()**
 - Initialization:
 - Start with **$z = \text{vec2}(0)$** , meaning the origin in the complex plane.
 - Iteration:
 - Iterate up to **maxIterations** times.
 - In each iteration, calculate **$z_{\text{Next}} = c + \text{complexPow}(z, \text{power})$** .
 - **If the magnitude of z_{Next} exceeds 2, break the loop. This indicates the point escapes the Mandelbrot set. Hint: $\text{Magnitude} = \text{length}(z_{\text{Next}})$**
 - Coloration:
 - Output a grayscale color based on whether the point escaped the Mandelbrot set (**$\text{vec4}(\text{color}, \text{color}, \text{color}, 1)$**).

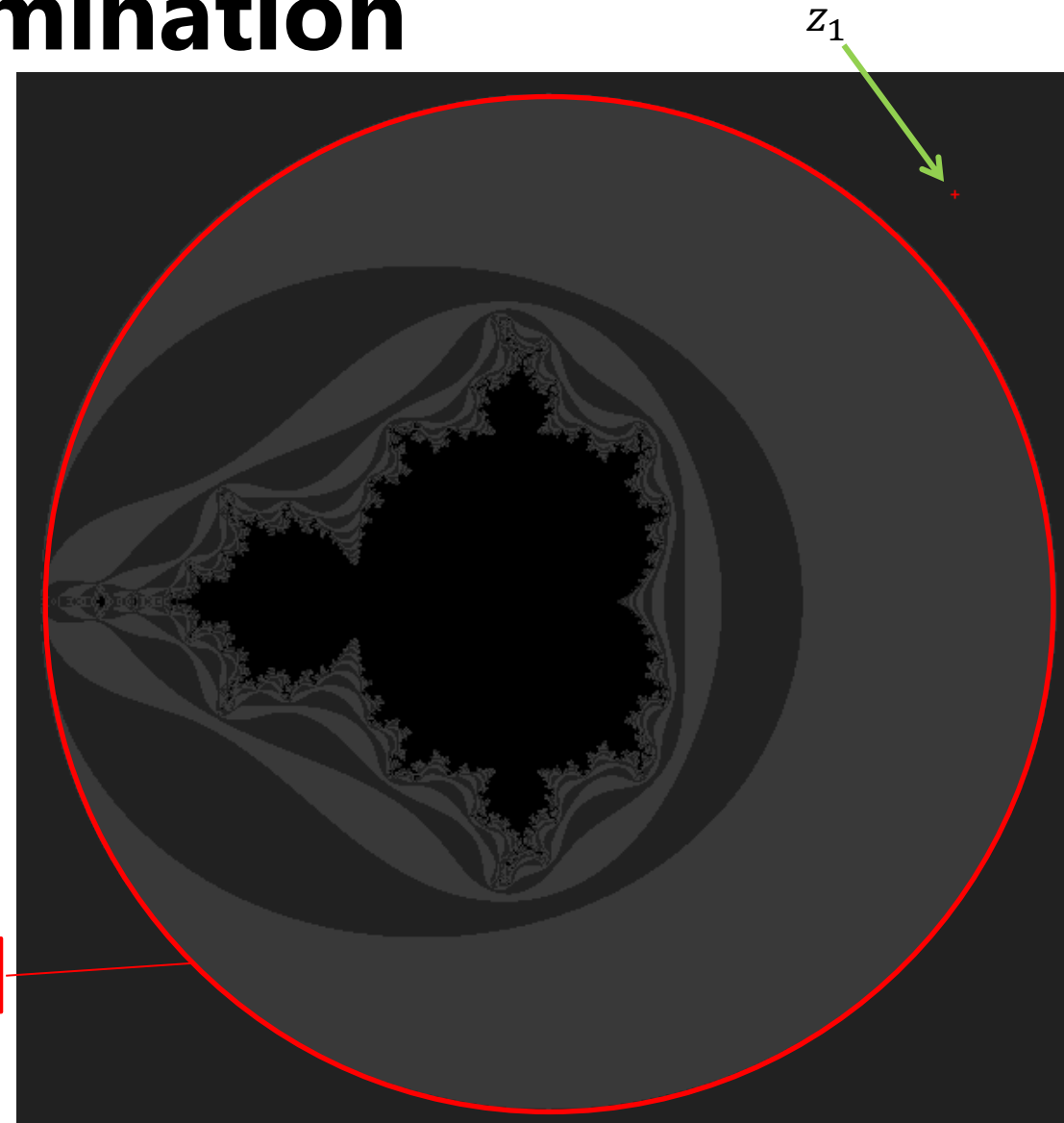
Example – Immediate Termination

$$z_1 = 1.6 + 1.6i$$

$$|z_1| = \sqrt{1.6^2 + 1.6^2} = 2.26 > 2$$

$\Rightarrow \text{STOP!}$

Circle of Radius 2 centered around the origin



Example – Early Termination

$$z_1 = 0.8 + 0.2i$$

$$|z_1| = \sqrt{0.8^2 + 0.2^2} = 0.84 \leq 2 \\ \Rightarrow \text{CONTINUE!}$$

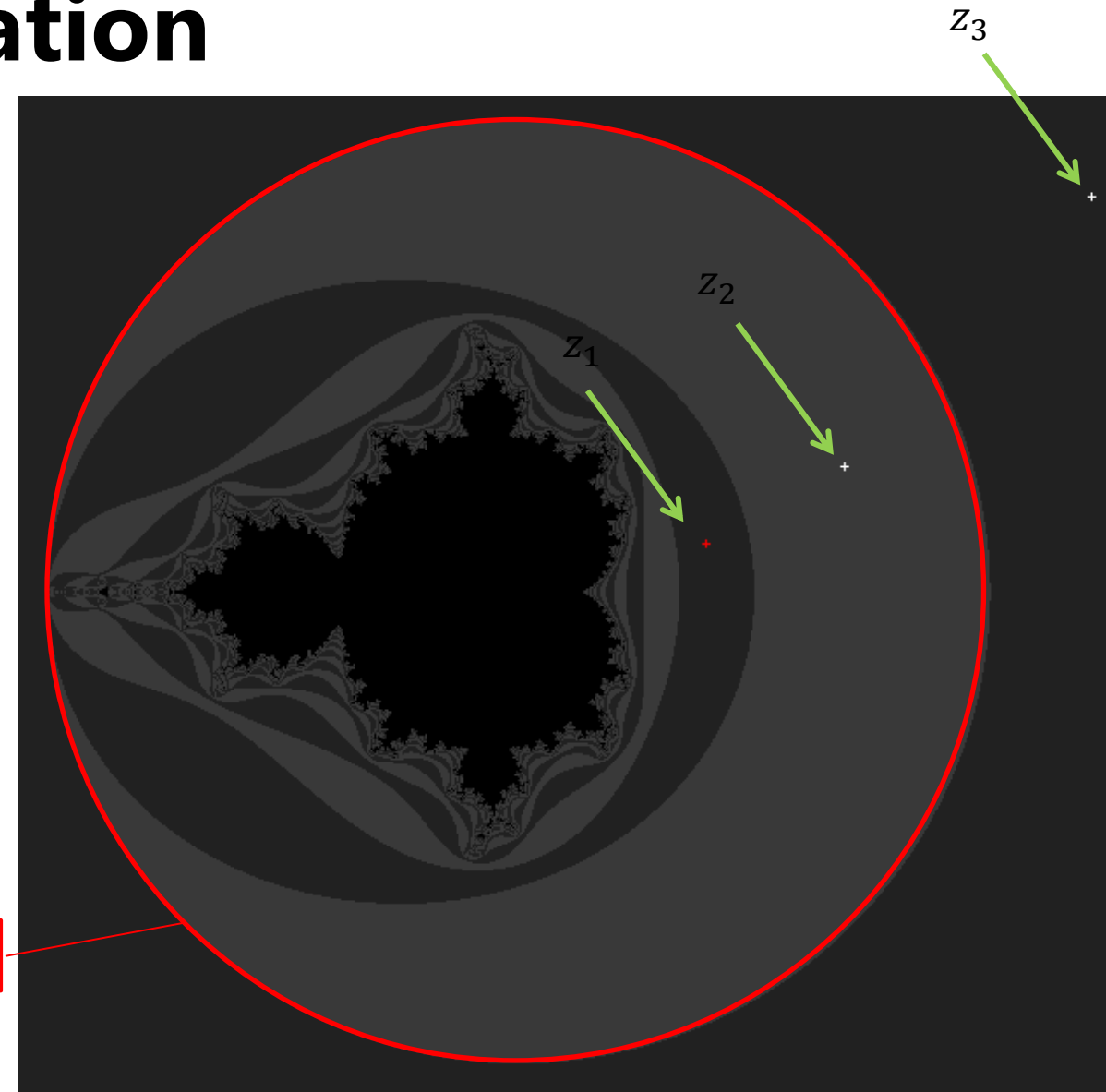
$$z_2 = 1.38 + 0.53i$$

$$|z_2| = \sqrt{1.38^2 + 0.53^2} = 1.47 \leq 2 \\ \Rightarrow \text{CONTINUE!}$$

$$z_3 = 2.42 + 1.67i$$

$$|z_3| = \sqrt{2.42^2 + 1.67^2} = 2.92 > 2 \\ \Rightarrow \text{STOP!}$$

Circle of Radius 2 centered around the origin



Example – Many Iterations

$$\begin{aligned} z_1 &= -0.4 + 0.05i \\ |z_1| &= \sqrt{(-0.4)^2 + 0.05^2} = 0.4 \leq 2 \\ &\Rightarrow \text{CONTINUE!} \end{aligned}$$

...

...

...

$$\begin{aligned} z_{20} &= -0.3 + 0.03i \\ |z_{20}| &= \sqrt{(-0.3)^2 + 0.03^2} = 0.3 \leq 2 \\ \dots &\Rightarrow \text{CONTINUE!} \end{aligned}$$

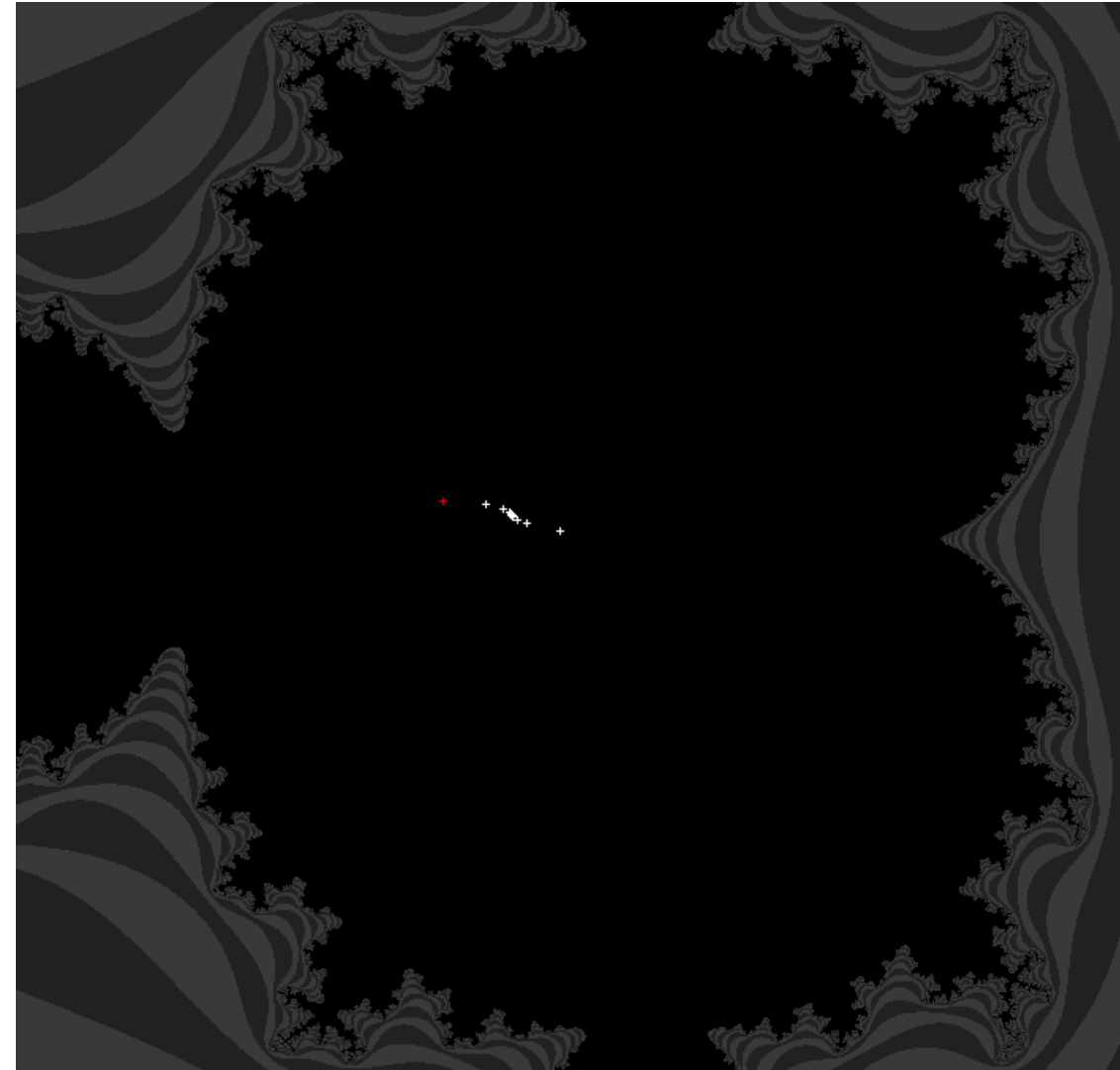
...

...

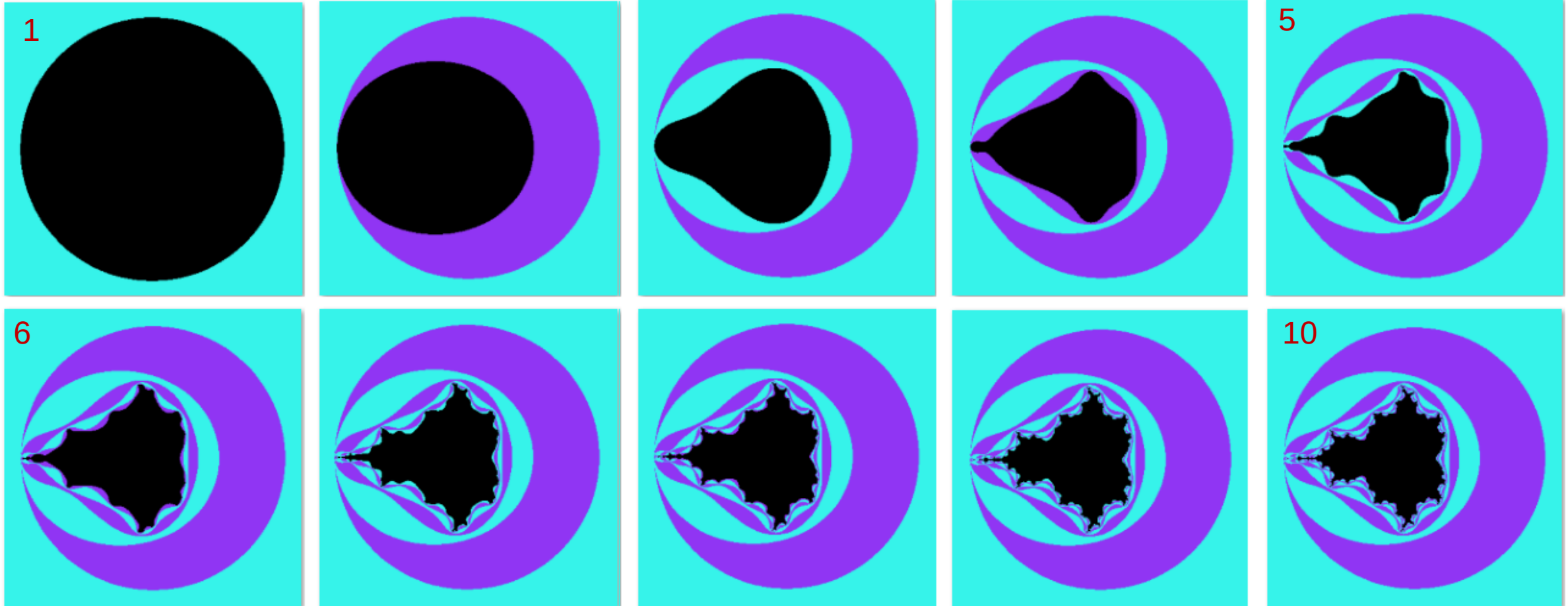
Video that shows “path” of calculated intermediate points:

“Sounds of the Mandelbrot Set” by CodeParade

<https://www.youtube.com/watch?v=GiAj9WW1OfQ>

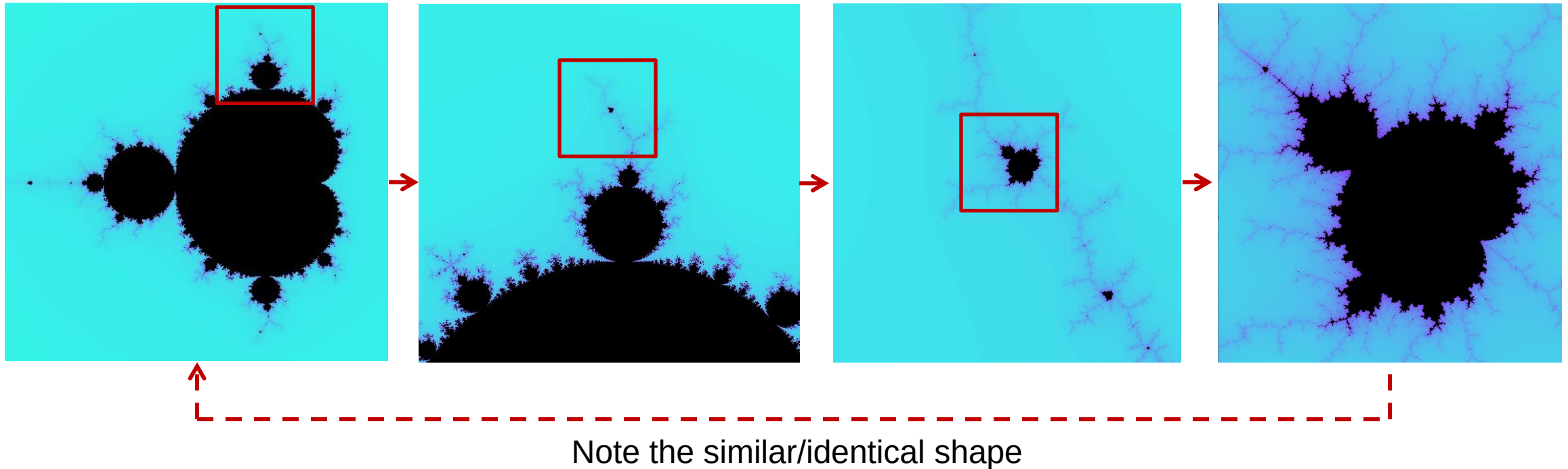


Approximate with finite Number of Iterations



Fractal Sets – Self Similarity

- Patterns re-appear (scaled/rotated) when zooming in



Follow the README 😊

**If anything is unclear:
Ask in Discord 😊**

OOP2 KU

Fractals, GLSL, LWJGL, GLFW and Java GPU Rendering

Sebastian David Albu, WS 2024

Based on slides from WS 2021 and WS 2022

Institute of Computer Graphics and Knowledge Visualization (CGV)